Aalto University
School of Science

Master's Programme in Computer, Communication and Information Sciences

Server-side rendering in web development

Toni Ojala

Master's Thesis
2025

© 2025

This work is licensed under a Creative Commons "Attribution-NonCommercial-ShareAlike 4.0 International" license.

Aalto University
School of Science

Author Toni Ojala

Title Server-side rendering in web development

Degree programme Computer, Communication and Information Sciences

Major Computer Science

Supervisor Prof. Petri Vuorimaa

Advisor M.Sc. Mikko Kiviniemi

Collaborative partner Ericsson

Date 30.06.2025 **Number of pages** 45 **Language** English

Abstract

In modern web development, rendering strategies have become a critical design choice, with server-side rendering emerging as an alternative or complement to client-side rendering. This thesis explores how SSR impacts web development by examining its core principles, the properties of SSR-capable JavaScript frameworks, its influence on the development process, and the feasibility of migrating existing applications from CSR to SSR. The study combines theoretical analysis with a migration example from CSR to SSR. A comparative evaluation of frameworks Next.js, Nuxt, SvelteKit, Angular, Astro, and Vike is presented, focusing on their architectural models, developer ergonomics, and other properties. The results indicate that while SSR introduces complexity in web development, modern frameworks ease the adoption by introducing conventions and utilities. The findings provide insights for developers and teams considering SSR, while also identifying critical areas that require careful attention during implementation. This research contributes to a deeper understanding of how rendering strategies affect application design, performance, and maintainability in a constantly evolving field.

Keywords web, software development, web application development, server-side rendering, rendering techniques, frameworks

Aalto-yliopisto
Perustieteiden
korkeakoulu

Tekijä Toni Ojala

Työn nimi Palvelinpuolen renderöinti websovelluskehityksessä

Koulutusohjelma Computer, Communication and Information Sciences

Pääaine Computer Science

Työn valvoja Prof. Petri Vuorimaa

Työn ohjaaja DI Mikko Kiviniemi

Yhteistyötaho Ericsson

Päivämäärä 30.6.2025 **Sivumäärä** 45 **Kieli** englanti

Tiivistelmä

Nykyaikaisessa web-kehityksessä renderöintistrategioista on tullut keskeinen suunnittelullinen valinta ja palvelinpuolen renderöinti (server-side rendering, SSR) on noussut vaihtoehdoksi tai täydentäjäksi asiakaspuolen renderöinnille (client-side rendering, CSR). Tämä opinnäytetyö tutkii, miten SSR vaikuttaa web-kehitykseen tarkastelemalla sen keskeisiä toimintaperiaatteita, SSR:ää tukevien JavaScript-kehysten ominaisuuksia, vaikutusta kehitysprosessiin sekä olemassa olevien CSR-sovellusten siirrettävyyttä SSR:n piiriin. Tutkimus yhdistää teoreettisen analyysin ja esimerkin CSR-sovelluksen migratoinnista SSR:ään. Työssä esitetään vertaileva arvio kehyksistä Next.js, Nuxt, SvelteKit, Angular, Astro ja Vike, keskittyen niiden arkkitehtuureihin, kehittäjäkokemukseen ja muihin ominaisuuksiin. Tulokset osoittavat, että vaikka SSR monimutkaistaa web-kehitystä, nykyaikaiset viitekehykset helpottavat käyttöönottoa tarjoamalla valmiita käytäntöjä ja työkaluja. Löydökset tarjoavat hyödyllisiä näkökulmia kehittäjille ja tiimeille, jotka harkitsevat SSR:n käyttöönottoa, ja tuovat esiin kriittisiä osa-alueita, joihin tulee kiinnittää huomiota toteutuksen aikana. Tämä tutkimus syventää ymmärrystä siitä, miten renderöintistrategiat vaikuttavat sovellusten suunnitteluun, suorituskykyyn ja ylläpidettävyyteen jatkuvasti kehittyvässä kentässä.

Avainsanat web, ohjelmistokehitys, websovelluskehitys, palvelinpuolen renderöinti, renderöintitekniikat, ohjelmistokehykset

Preface

I want to thank Professor Petri Vuorimaa, my instructor M.Sc. Mikko Kiviniemi, Jonas Lundqvist and Janne Hattula for their guidance and support during the thesis process.

I want to thank my family for their love and support during my life so far. Without you, I would not be here.

I want to thank my lovely fiancee Essi for always being there for me and supporting me unwaveringly during these precious seven years of studies. I am very lucky to share my life with you.

I want to thank my friends and fellow classmates for making the last 7 years such a blast.

Thank you JTMK'20, SIKH'21, FTMK'21, TPTMK'23, TPTMK'24, TJ'24, OJS, Paitsio, and Merikerho.

Thank you.

Espoo, 30.06.2025

Toni Ojala

Contents

Abstract **3**

Abstract (in Finnish) **4**

Preface **5**

Contents **6**

Terms and abbreviations **8**

1 Introduction **9**

2 Background **11**
2.1 Accessing a website 11
2.2 Rendering strategies 12
2.2.1 Client-side rendering (CSR) 12
2.2.2 Server-side rendering (SSR) 14
2.2.3 Static site generation (SSG) 14
2.3 JavaScript frameworks 16
2.3.1 React and Next.js 17
2.3.2 Vue and Nuxt 17
2.3.3 Svelte and SvelteKit 18
2.3.4 Angular 18
2.3.5 Astro 18
2.3.6 Vite and Vike 19
2.4 Related work 19

3 Research material and methods **21**
3.1 Research material selection 21
3.2 Methodological approach 21
3.3 Implementation and data collection 22
3.4 Methodological strengths and limitations 22
3.4.1 Strengths 22
3.4.2 Limitations 22
3.5 Usage of artificial intelligence 22

4 Results **23**
4.1 Impact on web development 23
4.1.1 Rendering strategy selection 23
4.1.2 Dual context development 24
4.1.3 Component design and data fetching patterns 24
4.1.4 State management and hydration 25
4.1.5 Debugging and error handling 26
4.1.6 Tooling and build pipeline complexity 26

4.1.7 Testing 27
4.1.8 Development team structure and collaboration 27
4.2 Frameworks 28
4.2.1 Key features and effects on development 28
4.2.2 Differences between frameworks 28
4.2.3 Framework vs. custom SSR solutions 30
4.2.4 Limitations of framework-based SSR 30
4.3 Migration 31
4.3.1 Migration checklist 33
4.3.2 Migration example from CSR to SSR 34

5 Conclusions **40**
5.1 Future work 41

References **42**

Terms and abbreviations

Terms

Application state	Data that an application manages and uses to determine its behavior at any given moment. It includes e.g. user inputs, UI status (e.g., whether a modal is open), authentication details, and fetched data.
Front-end	Client-side software
Back-end	Server-side software
Hydration	Process where client-side JavaScript takes over a server-rendered HTML page, attaching event listeners and restoring interactivity to make the page dynamic
Single page application	A website or a web application that dynamically rewrites the current view with new data instead of loading entire new pages.

Abbreviations

CDN	content delivery network
CSR	client-side rendering
FCP	first contentful paint
LCP	largest contentful paint
SEO	search engine optimization
SPA	single page application
SSG	static site generation
SSR	server-side rendering
TTFB	time to first byte
TTI	time to interactive

1 Introduction

Web development has evolved significantly over the recent years, with the rise of dynamic and interactive applications driving new challenges in performance, scalability, and user experience. One of the key techniques to address these challenges is SSR, a process in which web pages are generated on the server before being sent to the client. Server-side rendering (SSR) contrasts with client-side rendering (CSR), where the browser must download, parse, and execute JavaScript before rendering content, which can lead to slow initial page loads and high memory usage.

The shift toward JavaScript-based single-page applications (SPAs) has highlighted the limitations of CSR, particularly regarding search engine optimization (SEO), first-load performance, and browser resource consumption. SSR offers solutions to some of these problems by delivering pre-rendered HTML files to the client, leading to faster perceived performance and improved accessibility for users with limited computing power. However, SSR also introduces new complexities, such as increased server load, caching strategies, and the challenge of hydration, where client-side JavaScript must take over interactivity after the initial server-rendered content is loaded.

This study aims to explore SSR comprehensively, focusing on its fundamentals, viable frameworks and techniques, and its impact on the web development process. Several modern frameworks, including Next.js, Nuxt.js, SvelteKit, and Angular offer built-in SSR support, each with different trade-offs in performance, developer experience, and scalability. By analyzing these frameworks and techniques, this study will provide insight into how SSR influences development workflows and whether it can serve as a viable solution.

The primary research question that guides this thesis is:
How does server-side rendering impact web development?

To answer this, the study will address several subquestions:

- What are the key principles and techniques behind SSR?
- How do modern SSR frameworks compare with each other?
- How does SSR affect the web development process?
- Can web applications be migrated to use SSR and how?

This thesis will focus on JavaScript-based SSR implementations, given their widespread usage in modern web development. The study does not cover traditional SSR approaches in non-JavaScript ecosystems, such as PHP or Ruby on Rails. The findings aim to provide theoretical and practical information that will benefit developers, researchers, and organizations seeking to optimize the performance of

web applications and user experience.

By examining the capabilities and limitations of SSR, this thesis will contribute to the ongoing discussion of rendering strategies in web development and explore whether SSR offers a sustainable solution to browser performance bottlenecks.

In the background section, this study introduces the technologies behind modern web software, different rendering strategies, and the most prominent frameworks to implement SSR with. In research material and methods, the chosen research material, methodological approach, implementation, and methodological strengths and limitations of the study are explored. In results, the effects of SSR to web software development are studied, a comparison of the chosen frameworks is conducted based on the chosen criteria, and the migration from CSR to SSR are studied. Lastly, the findings of this study are summarized in conclusions.

2 Background

This section describes the process of what happens when a user accesses a website, how the website is rendered, and lays out the details of the rendering process of different web rendering techniques.

2.1 Accessing a website

When a user navigates to a website, the browser first has to find out where the server of the domain is located by resolving the input domain to an IP address. This is done by doing a Domain Name System (DNS) lookup. The browser checks its cache to see if it already knows the IP address of the domain. If the address is not found, the browser prompts the operating system to resolve the domain. The operating system then queries a DNS resolver (usually provided by the internet service provider or a public DNS service like Google Public DNS). The DNS resolver looks up the domain name and returns the corresponding IP address to the user. The browser stores the location for a set amount of time, which varies between servers. [5]

After getting the resolved IP address, the browser initiates a Transmission Control Protocol (TCP) handshake with the server, which follows a three-step process. The client first sends an SYN packet to the server to initiate the connection. The server responds with a SYN-ACK packet. Finally, the client sends an ACK packet, completing the handshake. In case Hypertext Transfer Protocol Secure (HTTPS) is used, an additional Transport Layer Security (TLS) handshake will also take place. The TLS handshake begins with the client requesting a secure connection. The server then provides an SSL/TLS certificate to prove its identity, and a secure key exchange occurs to encrypt the communication. [5]

Once the handshakes have been completed, the browser proceeds to send an HTTP request (or HTTPS if using TLS) to the server. The request typically includes the HTTP method (e.g., GET, POST), headers (such as user-agent, cookies, or authentication tokens), request URL path (e.g., /home) and some additional data if it is a POST request. The server then processes the request sent by the browser. In case of a static website, the server directly serves the browser the HTML, CSS, JavaScript, and media files. In case of a dynamic website, the server might also query a database, execute server-side code, and generate a response based on the user authentication or business logic. The server then sends back the HTTP response that includes the HTTP status code, the requested content (for a website this is the HTML page with possible CSS and JavaScript) and headers such as Content-Type, Cache-Control, or Set-Cookie. [5]

After receiving the response, the browser starts processing the HTML file. It parses the HTML file and constructs the Document Object Model (DOM) accordingly. The Document Object Model is a programming interface for web documents, which represents the page so that programs can change the document structure, style, and

content. The DOM represents the document as nodes and objects; in this way, programming languages can interact with the page. [5][9]

If the HTML references external resources (like stylesheets, fonts, scripts), the browser makes more HTTP requests to load them. After loading the resources, the browser applies the Cascading Style Sheets (CSS) styles to the website and executes JavaScript scripts. If there are images or other resources, the browser sends additional requests to fetch them. Additional resources may come from the same server (first-party resources), a CDN (Content Delivery Network), or third-party servers (for ads, analytics, APIs). After the browser has combined the DOM, CSS, and JavaScript, it proceeds to render the page visually. Any client-side scripts (like interactive elements, animations) run at this stage. If needed, JavaScript may make additional requests to load more data asynchronously. The website is then visually presented to the user by the browser. [5]

As described, a website can consist of multiple different resources and, therefore, can have a quite complicated structure. The structure of the HTML file is a significant factor regarding performance, and is one of the main aspects the rendering techniques differ from each other.

2.2 Rendering strategies

With websites comprising of multiple parts, building and rendering the whole package can be done in multiple ways. Here are the most common rendering strategies and techniques used in modern web software.

2.2.1 Client-side rendering (CSR)

CSR is a web development technique where the browser generates the HTML content using JavaScript, as opposed to receiving pre-rendered HTML from the server. In this approach, the server typically delivers a minimal HTML page along with the necessary JavaScript files to render the page. Once these scripts are downloaded and executed, the browser dynamically constructs the DOM and renders the content. [10][11]

CSR is widely adopted in applications with dynamic content and a strong emphasis on interactivity, such as chat applications and social media platforms. It is very well-suited for SPAs and internal tools like admin panels and user dashboards, where search engine indexing is not a priority.

Advantages of client-side rendering include:

- Improved interactivity: CSR enables the development of highly interactive applications, as it allows for dynamic updates to the user interface without requiring full page reloads. [12]

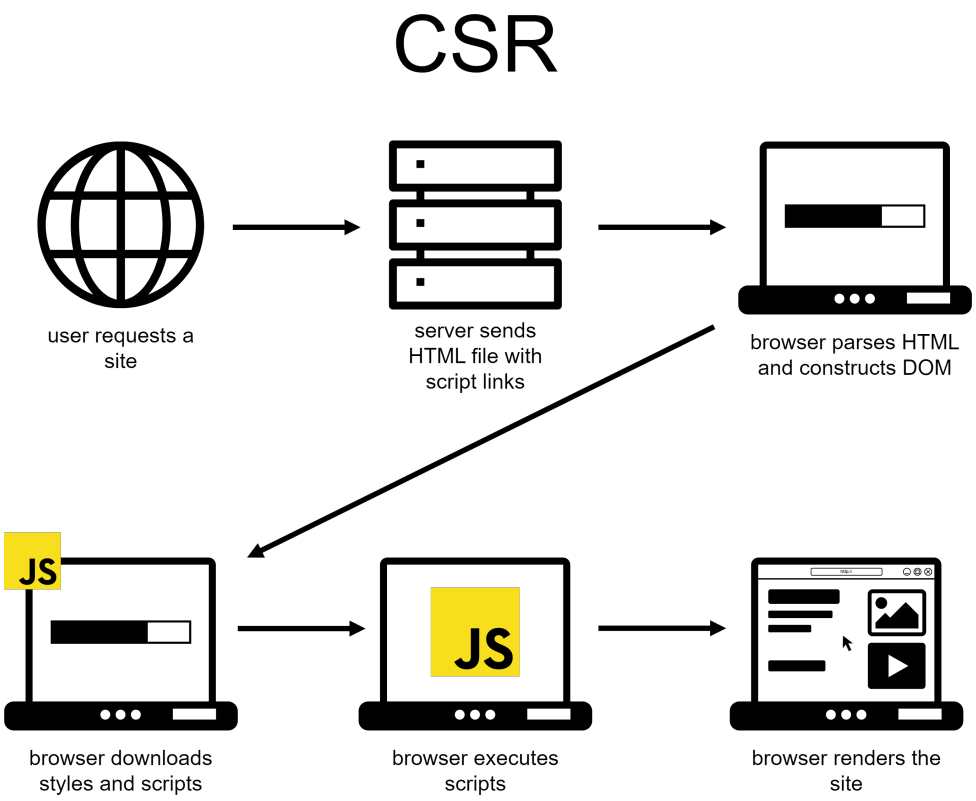

Figure 1: Illustration of user accessing a website using CSR.

- Reduced Server Load: By offloading the rendering process to the client's browser, CSR can decrease the computational burden on the server, potentially leading to improved scalability. [12]

Challenges associated with client-side rendering include:

- Initial load performance: Since the browser must download and execute JavaScript before rendering content, users might experience longer initial load times, especially on slower networks or devices. [11]
- SEO: Some search engines may encounter difficulties indexing content that is rendered client-side, potentially impacting the site's visibility in search results. [13]

CSR is commonly used in SPAs, where the goal is to provide a seamless and responsive user experience by dynamically updating the content without full page refreshes. Although CSR offers benefits such as improved interactivity and reduced server load, it also presents challenges, such as initial load performance and taking SEO into account. Strategies to mitigate these issues, such as code splitting, SSR, or hybrid approaches that combine CSR and SSR techniques, may need to be taken into account in planning. [12][14]

2.2.2 Server-side rendering (SSR)

SSR is a web development technique, in which HTML content is generated on the server and sent to the client, compared to client-side rendering, where the browser constructs the HTML using JavaScript. The generated HTML content can then be hydrated with JavaScript to handle interactivity. [15] One major difference from CSR and SSG is that SSR must have a server to operate, which is not necessary for the other two methods. A Node.js-based server is often used in SSR applications built with modern JavaScript frameworks.

Advantages of SSR include:

- Improved performance: Since the server sends fully rendered HTML, users often experience faster initial load times, especially on devices with limited processing power or slower network connections. [15]
- Improved SEO: Search engines can more effectively index content that is rendered server-side, potentially improving the site's visibility in search results. [15]

Challenges associated with SSR include:

- Increased server load: Rendering content on the server for each request can lead to higher computational demands, especially for applications with dynamic content. [15]
- Slower subsequent page loads: Rendering the page on the server can result in slower subsequent page loads if the client needs to make additional server requests, which can be caused by interactivity.
- Complexity in state management: Maintaining application state between the server and client can be more challenging, requiring additional mechanisms to synchronize data. [15]

SSR is particularly beneficial for applications where quick initial load times and SEO are critical, such as e-commerce sites. However, trade-offs must be considered when using SSR. [15]

2.2.3 Static site generation (SSG)

SSG is a web development technique where HTML pages are pre-built at build time, rather than being rendered dynamically on the server or client at runtime. This approach involves using a static site generator to compile data and page templates into fully rendered HTML pages ahead of time. Static sites often do not need to be deployed on a server at all, and can be served from a CDN. [13][15][16]

Advantages of static site generation include:

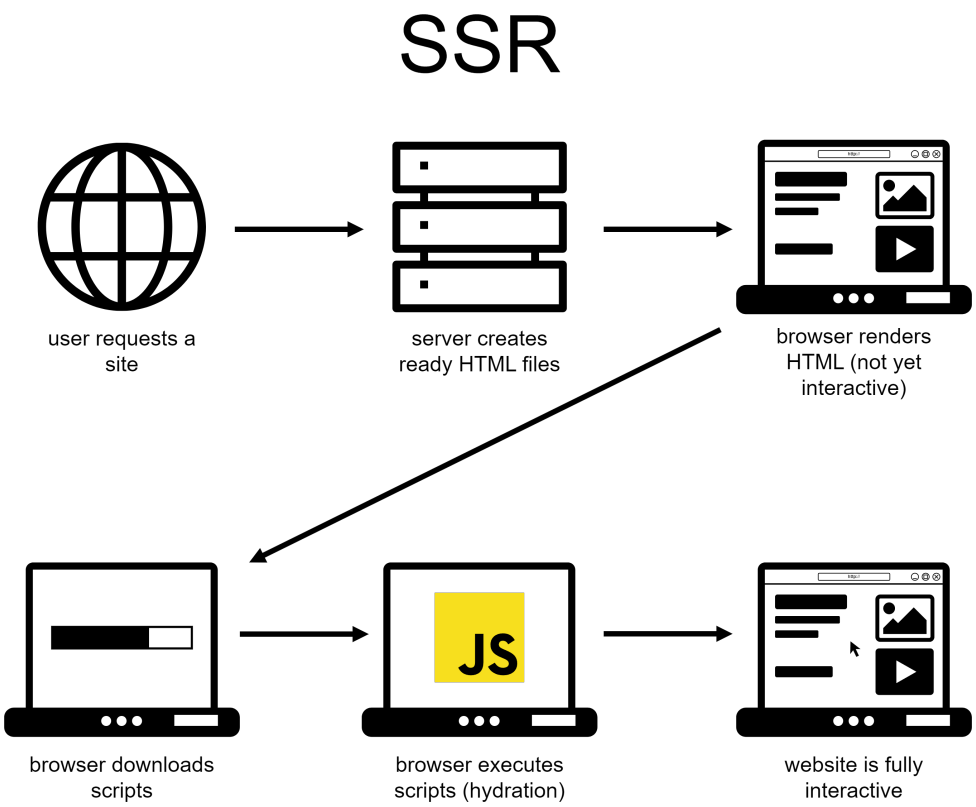

Figure 2: Illustration of user accessing a website using SSR.

- Improved performance: As the HTML pages are pre-rendered and served as static files, they can be delivered to users more quickly, resulting in faster load times. [13][19]
- Scalability: Static sites can be easily served through CDNs, enabling efficient scaling to handle large volumes of traffic.

Challenges associated with static site generation include:

- Content updates: Updating content requires regenerating the static files and redeploying them, which may not be ideal for sites with frequently changing content. [18]
- Lack of dynamic functionality: Out of the box, static sites lack features like user authentication or real-time data fetching, which require additional configurations or integrations to achieve such functionalities. [18]

Static site generation has become an increasingly important option in modern web development, especially within the Jamstack architecture, which emphasizes decoupling the frontend from backend to improve performance and security. [18]

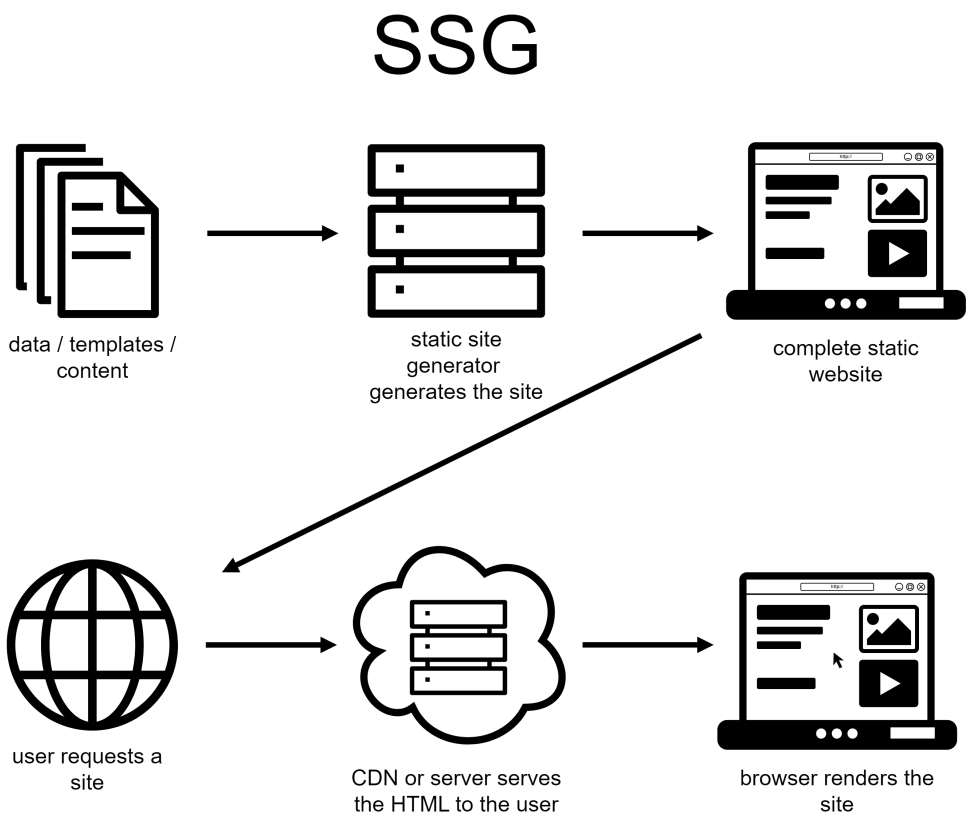

Figure 3: Illustration of user accessing a website using SSG.

2.3 JavaScript frameworks

JavaScript frameworks play an important role in modern front-end web development, offering developers reliable tools to create scalable and interactive web applications. As the usage of frameworks and libraries have become a standard in the industry, many companies expect their front-end developers to have experience working with them. [20]

In the early days of JavaScript, developers who worked with it wrote tools to solve problems they faced, and packaged their scripts into reusable packages called libraries, so their solutions could be shared with others. The ecosystem of shared libraries shaped the growth of web software and eventually gave way to frameworks. [21]

A framework is a library that offers opinions about how software is developed and built. These opinions allow for predictability and uniformity in an application. Predictability allows the software to scale and still be maintainable. Predictability and maintainability are essential for the longevity of software. The rise of modern JavaScript frameworks has made it easier to build highly dynamic and interactive applications. This is much due to the verbosity of DOM-related updates and changes, which the frameworks do for the developer. [21]

Compared to direct DOM manipulation via JavaScript, the JavaScript frameworks offer a way to write websites and user interfaces more declaratively. In other words, they allow you to write code that describes how the interface should look, and the framework manipulates the DOM accordingly behind the scenes. [21]

The frameworks introduced and compared in this study are selected based on their popularity, maturity, and server-side rendering support. This is why very recent and promising frameworks such as SolidJS and Qwik are omitted from the list, as they are yet to prove their longevity in this field. For each introduced framework/library, an additional framework or library that is built on top of it is introduced. The reason for this is that most frameworks recommend using a more higher-level framework when implementing software that wants to utilize different rendering strategies or have more control over it (like React, Vue, and Svelte do) [22][29][33].

2.3.1 React and Next.js

React is a JavaScript library for rendering user interface components, developed by Facebook and released in 2013 [21][22]. React is often used for single-page applications where interactive user experience is essential. It enables developers to create reusable UI components that manage their own state, leading to efficient and predictable rendering of interfaces. React's declarative approach simplifies the process of describing and designing interface views, making sure that the UI reflects the current state of the application. [23][21] React uses a markup syntax called JSX to describe components in an HTML-like way [22][24]. Although React is a library, as React and ReactDOM are so often used together, React is colloquially understood as a JavaScript framework [21].

Next.js is a React framework for building full-stack web applications introduced by Vercel in 2016. React components are used to build user interfaces in Next.js applications, but Next.js offers additional features and optimizations on top of that. In addition to CSR, which is typical for React applications, Next.js offers different rendering strategies, such as SSR, SSG, and hybrid options. Next.js abstracts the configuration required for setting up a React application and automatically configures the tooling needed, like bundling, compiling, and more. It also offers code splitting and routing out-of-the-box. [25][26]

2.3.2 Vue and Nuxt

Vue.js is a progressive JavaScript framework for building user interfaces, developed by Evan You and released in 2014. It is built on top of standard modern HTML, CSS and Javascript, extending their basic functionalities. Vue offers a declarative, component-based programming model. [21][27]

Nuxt is a Vue framework designed to enhance the development of full-stack Vue.js

applications. It provides an intuitive and extendable architecture for creating type-safe, performant, and production-grade full-stack web applications and websites. [28] Nuxt implements better utilities and control for different rendering strategies than plain Vue, although Vue has built-in support for, e.g., SSR. The use of a higher-level solution like Nuxt is recommended by Vue. [29]

2.3.3 Svelte and SvelteKit

Svelte is a framework for building user interfaces on the web, developed by Rich Harris and released in 2016 [30][31]. It uses a compiler to turn declarative components written in HTML, CSS and JavaScript into a lean format [30]. Svelte uses a markup syntax that is a superset of regular HTML [32].

SvelteKit is a framework for building web applications using Svelte. SvelteKit provides an extensive list of features in addition to regular Svelte, such as build optimizations to load only the minimal required code, offline support, preloading pages before user navigation, configurable rendering to handle different parts of your app on the server via SSR, in the browser through CSR, or at build-time with prerendering and image optimization. [33] SvelteKit uses SSR as its default rendering strategy, but offers both CSR and SSG options as well [34].

2.3.4 Angular

Angular is a JavaScript framework for developing web applications, developed by Google and released in 2016 [21]. It is a complete rewrite of its predecessor AngularJS, made by the same team of developers [21]. Angular offers a component-based architecture that improves modularity and maintainability. It provides an extensive suite of tools and libraries, including features such as two-way data binding, dependency injection, and a robust routing system, enabling creation of scalable and efficient web applications. [35]

Angular Universal is a prerendering technology for Angular that enables SSR and hybrid rendering strategies for Angular applications [37]. As of version 17, the technology has been adopted to a separate Angular package called SSR. In addition to Angular Universal's features, SSR package added partial hydration, which allows for more control over hydration, as a new feature. [38] SSR can be added to any Angular application, providing the benefits of SSR without requiring a complete overhaul of the existing codebase [36][37][38].

2.3.5 Astro

Astro is a server-first JavaScript framework designed for building content-driven websites like blogs, marketing, and e-commerce developed by Fred K. Schott and released in 2021. Astro is known for pioneering a new front-end architecture to reduce JavaScript overhead and complexity compared to other frameworks. It combines the

benefits of SSG and SSR to create performant and SEO-friendly applications. Astro is also framework-agnostic, meaning that you can use UI components built with other frameworks (such as any of the frameworks introduced in this study) and even mix them, making it a unique choice among frameworks. [39][40][41]

Astro generates static sites by default, but also has built-in SSR functionality via SSR adapters for Node.js, Netlify, Vercel, and Cloudflare. Each adapter allows Astro to run on a specific runtime environment that runs code on the server to generate pages when they are requested by users. It is possible to add an adapter to an existing Astro project and convert it to use, e.g., SSR that way. [42] Astro also has component-specific hydration control to make components interactive when using UI components built with another framework. This is done by passing an attribute to the component, which determines when the component's JavaScript should be sent to the browser (such as on page load or when the component is visible in the browser). [40]

2.3.6 Vite and Vike

Vite is a build tool and development server, developed by Evan You (creator of Vue.js) and released in 2020. Its design improves the developer experience by enabling near-instant hot module replacement (HMR) and fast cold starts, even in large projects. Vite supports multiple frameworks and libraries, and functions as a scaffolding software. [43]

Vike (formerly known as vite-plugin-ssr) is a modern meta-framework built on top of Vite that provides a flexible foundation for implementing SSR in JavaScript and TypeScript applications. Unlike opinionated frameworks, Vike follows a minimalist, unopinionated design philosophy, giving developers control over routing, data fetching, and rendering behavior. This makes it suitable for projects that require custom SSR logic without sacrificing the performance and developer experience benefits of Vite, or smaller projects that require only simple basic functions. Vike allows developers to define per-page configuration files and rendering strategies, enabling hybrid rendering approaches similar to those found in more complex frameworks, without imposing as much conventions or folder structures. It supports multiple front-end libraries and frameworks (e.g., React and Vue) and encourages separation of concerns by making explicit distinctions between server-side and client-side code. By embracing modularity and letting developers opt into features as needed, Vike offers a middle ground between implementing a custom SSR pipeline and adopting a fully opinionated meta-framework. [44]

2.4 Related work

An article by Nikhil Sripathi Rao offers a technical deep dive into modern SSR, emphasizing its benefits for performance metrics such as FCP and LCP compared to CSR, particularly on mobile and bandwidth-constrained devices [2]. Similarly, a study by Karthik Vallamsetla documents how frameworks like Next.js, Nuxt, and

Angular Universal simplify SSR implementation while delivering SEO and performance gains [3].

The article *The State of Disappearing Frameworks in 2023* (that is a part of Juho Vepsäläinen's doctoral thesis) introduces the concept of "disappearing frameworks" like Astro, which deliver minimal JS to the client through progressive enhancement and static output. They articulate the rising trend toward partial hydration and content-first architectures, an area explored in this study through Astro and Vike. Otherwise, the doctoral thesis covers many of the concepts covered in this study. [1]

Aleksi Huotala's master's thesis presents a developer-centered study on isomorphic architectures, combining SSR and CSR. It highlights how SSR improves code reuse and maintainability, but introduces challenges such as increased architectural complexity and testing overhead. It provides insights into how the adoption of SSR affects the workflow of developers and the structure of the project, aligning closely with the focus of this thesis on the development process. [4]

Jaakko Soitinaho's master's thesis focuses specifically on the practical feasibility and challenges of migrating a legacy CSR-based enterprise SPA to a server-side renderable application. The thesis is particularly aligned with the migration aspects discussed in Section 4.3 of this work. [8]

Most related academic work focuses on technical or system-level metrics rather than developer-centric process implications. This thesis aims to fill a gap by examining how SSR affects the development process, including migration strategies from CSR to SSR, and providing analysis of modern SSR tools like Next.js, Nuxt, SvelteKit, Angular, Astro, and Vike.

3 Research material and methods

This study uses a combination of theoretical analysis and empirical evaluation to investigate the role of server-side rendering in web development. The research follows a qualitative and quantitative approach, consisting of literature review, comparative analysis of SSR frameworks, and practical experimentation to evaluate migration from CSR to SSR.

3.1 Research material selection

The research material consists of the following:

- Academic literature and technical documentation: Books, research papers, and technical articles on rendering strategies and web development.
- Official framework documentation: Materials from developers of modern SSR frameworks, which outline implementation details and best practices.
- Reports and case studies: Analyzes conducted by third parties, including comparisons of SSR and CSR.

3.2 Methodological approach

The study is divided into three methodological components:

- Literature review
  - A review of existing research on rendering techniques, SSR frameworks, trade-offs between SSR and CSR, and effects of SSR to web development processes.
- Comparative analysis
- A comparative evaluation of Next.js, Nuxt.js, SvelteKit, Angular, Astro, and Vike focusing on their characteristics, developer experience, and suitability for different use cases.
- Implementation and empirical testing
  - Frameworks are analyzed by, e.g. creating new projects with them to gain an understanding of their conventions and characteristics.
  - A test application will be built using Vike to demonstrate development and migration from CSR to SSR.

3.3 Implementation and data collection

- To ensure reproducibility and accuracy, the experimental setup will follow these principles:
  - Controlled environment: The test application will be run on identical hardware and network conditions (as in with the same computer).
  - Multiple rendering scenarios: The application will be tested under different rendering strategies, CSR and SSR.

3.4 Methodological strengths and limitations

3.4.1 Strengths

- Comprehensive scope: The combination of literature review, framework analysis, and experimental validation provides a holistic view of the impact of SSR.

3.4.2 Limitations

- Framework-specific results: The findings may be biased towards the specific SSR frameworks chosen for the analysis.
- Scope restriction: The study focuses primarily on JavaScript-based SSR frameworks, excluding traditional SSR implementations in other programming languages.
- Uncertainty based on implementation: The potential effects of SSR on development are uncertain due to the relation to developer knowledge and lack of implementation details.
- Lack of extensive testing in real-world software development: Creating enterprise-grade applications with each of the selected frameworks is not feasible within the scope of this study.

By combining theoretical insights with limited empirical data, this study aims to provide a practical evaluation of SSR's role in modern web development, offering insight into its benefits and trade-offs.

3.5 Usage of artificial intelligence

Artificial intelligence has been used in this thesis as a tool for proofreading and structuring text.

4 Results

4.1 Impact on web development

SSR represents a significant shift in the way modern web applications are built, deployed, and experienced. Its adoption across popular web development frameworks has highlighted both its benefits and trade-offs. This subsection explores SSR's implications in web development across multiple dimensions. SSR represents more than a performance optimization strategy; it significantly changes the way web software is developed, structured, and maintained. The adoption of SSR introduces changes to development workflows, application architecture, state management strategies, testing paradigms, and developer responsibilities.

The following results are primarily compiled on the basis of studying related work and earlier research, as well as cursory testing of the selected frameworks.

4.1.1 Rendering strategy selection

Having SSR as a rendering strategy option makes it possible to render web software according to the needs of the application. The ability to choose between CSR, SSR, or a hybrid approach affects the landscape of web software development by introducing flexibility, performance optimization opportunities, and architectural options. CSR, where rendering is handled in the browser, offers highly dynamic and interactive user experiences, but can suffer from slow initial page loads and SEO challenges, especially in content-driven applications. In contrast, SSR generates HTML on the server, enhancing the first contentful paint and search engine visibility, which is crucial for content-heavy or marketing-focused sites. The hybrid approach, which is seen in some frameworks, allows developers to strategically combine the strengths of both paradigms by enabling SSR for pages that need SEO or quick load times, while offloading interactive components to the client for smoother user interactions after loading. This enables developers to tailor performance and user experience on a per-page or even per-component basis, which aligns with the needs of modern applications where parts of the interface may need prioritize different concerns such as latency, personalization, or interactivity. The shift towards flexible rendering paradigms reflects the broader evolution of web software development from static documents to dynamic, user-centered applications where performance, accessibility, and scalability can be finely tuned based on the needs of the solution. [1][2][3]

However, having so many options to choose from can result in a more complex and potentially stressful situation at the beginning of a new project. When aiming for a fitting solution, many details need to be taken into account when choosing a rendering strategy. Should the development team end up choosing a framework for developing the application, they need to be able to make sure the framework is capable of the desired rendering strategy and has the utilities to potentially extend upon that base, such as aforementioned per-page or per-component rendering strategy control.

Another option would be to opt to develop one's own custom solution to have maximal control and no framework dependency, but this would require a lot of work and skills from the developing party. In either case, the introduction of SSR as an option will lead to an increased demand for developer skills and knowledge about rendering strategies as a whole. [1][2][4][5]

4.1.2 Dual context development

SSR introduces a dual context environment, where code can be executed both on the server and on the client, affecting the development, structure, and complexity of web software. This also implies that the application state is fragmented into two, which means that the two states need to be in sync during runtime of the software. This duality challenges traditional assumptions about runtime behavior, demanding that developers consider the execution context at different stages of the application lifecycle. In SSR, components or logic intended to run on the server at request time must avoid reliance on browser-specific APIs like window, document, or local storage, while client-side interactivity still depends on these very constructs. This makes it necessary to take into account careful architectural separation or context-aware coding patterns, such as guards or hydration-aware hooks, to ensure the code behaves correctly regardless of where it runs. [1][2][4][5]

The implications extend to data fetching, authentication, and even error handling. Functions may need to be duplicated or abstracted to handle both server and client use cases. Additionally, managing state across these two contexts becomes non-trivial, especially when initial server-rendered HTML must synchronize with client-side scripts during hydration, which is a phase prone to mismatches or flickers if not handled properly. Performance trade-offs introduced by SSR must also be taken into account. While it can reduce time-to-first-byte and improve SEO by prerendering content, it may increase server load and latency if not paired with caching strategies like CDN-based edge rendering or ISR (Incremental Static Regeneration). The dual-context nature of SSR forces a more disciplined approach to code organization and modularity, encouraging patterns such as universal (isomorphic) JavaScript and rendering-aware abstraction layers. Ultimately, while SSR offers powerful advantages for speed, SEO, and accessibility, its dual-execution model introduces complexity that affects the development workflow, tooling, and mental model. It requires teams to pay more attention to detail in their design and engineering practices and requires more knowledge from each developer. [1][2][3][4][5][6]

4.1.3 Component design and data fetching patterns

Component architecture and data flow are affected by SSR. While traditional front-end components are reactive and client-sided, SSR enforces a separation between rendering and interactivity. This is particularly evident in some of the SSR frameworks, which introduce server-oriented data fetching methods to retrieve and inject data during the rendering phase. This kind of method is often used with Next.js

or Nuxt [25][28].

Using SSR requires developers to structure their applications to distinguish between different key aspects such as:

- Data needed for initial rendering (retrieved on the server)
- Data fetched post-hydration (retrieved on the client)
- Data shared between both contexts (often cached or rehydrated)

[2][3][10][15]

This separation increases the cognitive load, as logic that is confined to the browser in CSR must now be abstracted across multiple layers of software. Developers must consider performance trade-offs associated with each rendering strategy, balancing time-to-first-byte (TTFB) or time-to-interactive (TTI), SEO considerations, and user interactivity. This is often determined by the application at hand. [2][3][4][5]

Frameworks like Astro take this further by having a component-islands architecture, where static pages are rendered server-side, and only select interactive components are hydrated. Such patterns require developers to make granular architectural decisions about what should be rendered where, potentially making the process more complex even further while allowing for more control over the behavior of the software. [1][2][39][44]

4.1.4 State management and hydration

State management under SSR introduces additional complexity due to the need to synchronize state across the server and client. In CSR, application state resides entirely in the browser and is initialized once. In SSR, state must often be:

- Fetched and assembled on the server
- Serialized and embedded in the HTML (dehydration)
- Parsed and rehydrated on the client during page load

This process raises potential security and performance concerns. Sensitive server-side data (such as user tokens, credentials, and internal logic) must be carefully handled to avoid unintentional client exposure in HTML response, while serialization overhead and hydration logic can introduce runtime latency. [2][3][15][48]

To address these challenges, developers can:

- Use SSR-compatible global state libraries (e.g., Redux with custom middleware, Vuex with SSR plugins)

- Carefully scope stateful logic to components that are hydrated
- Utilize data-fetching libraries with hydration-aware APIs to avoid issues

These new requirements impact how developers design data flows, handle loading states, and avoid duplicate API calls on both the server and the client to make the software run smoothly. [2][5][28][29][47]

4.1.5 Debugging and error handling

Debugging SSR applications introduces new challenges due to the dual nature of server and client execution compared to normal modern CSR web applications. When typically errors happen only locally in the client software or in the server software (e.g., backend APIs), with SSR the errors can happen in more locations and ways. In SSR, the errors may:

- Occur only during server rendering (e.g., ReferenceError: window is not defined when trying to reference client-only APIs)
- Arise during hydration when HTML mismatches between the server and client
- Manifest only in dynamic client-side updates after hydration

[4][9][25][27]

This can make it more difficult to pinpoint the cause of the error. Therefore, developers must know how to execute distinct debugging strategies in order to solve the issues:

- Server logs and error monitoring
- Client-side debugging tools like React/Vue/Svelte/Astro dev tools
- Hydration warnings and mismatch detection mechanisms

This added complexity makes testing and error diagnosis more time-consuming than normally and requires a deeper understanding of the framework internals and rendering lifecycles. [4][49]

4.1.6 Tooling and build pipeline complexity

SSR changes the build and deployment process by introducing the need for dual compilation targets. One is needed for SSR and another for client-side interactivity. Frameworks automate this to an extent, but developers must still be familiar with server-side bundlers (e.g., Webpack, Vite, ESBuild), environment variable scoping (e.g., process.env vs. window.ENV), and output separation (such as server entry files vs. client bundles). Build performance, server cold start times, and bundling errors may surface due to misconfigured dependencies or non-isomorphic code. This

indicates on a requirement for a stronger DevOps awareness among front-end developers, further blurring the traditional lines between frontend, backend, and DevOps responsibilities in software development teams, putting more emphasis on full-stack mentality among developers. [4]

Additionally, depending on the deployment strategy, SSR deployments might also require integration with edge networks or serverless runtimes (e.g., Vercel, Netlify, Cloudflare Workers), which might add further constraints to how code is written and shipped. This potentially affects larger companies with more proprietary software infrastructure less as they probably don't need to rely on 3rd party environments. [2][4]

4.1.7 Testing

As SSR software introduces more phases of execution and software delivery, testing is obviously also affected. More states need to tested to ensure correct behavior as the software is shipped, as accessing the client software now includes rendering on the server as an extra step. Testing SSR applications introduces the need to test multiple phases that include:

- Pre-hydration tests: Need to ensure that, for example, the HTML structure and the SEO tags are correct before hydration.
- Hydration tests: Need to validate that interactivity is correctly initialized after delivering the prerendered HTML.
- Post-hydration tests: Need to confirm that client logic behaves as expected.

End-to-end testing frameworks such as Cypress and Playwright support SSR testing, but require careful setup to simulate full server-to-client transitions. Moreover, snapshot testing must distinguish between static HTML and hydrated DOM trees, complicating test assertions. This requires more thorough test coverage and environment-specific mocking, increasing the overall testing burden in SSR projects. Lack of research and material online indicates, that this section of SSR is still understudied.

4.1.8 Development team structure and collaboration

SSR adoption can also drive organizational change. Traditional front-end developers must now interface more closely with backend teams, to coordinate API access and server-rendered data properly. They must also collaborate with DevOps teams, to deploy and monitor SSR infrastructure properly. This encourages a full-stack mindset and cross-functional collaboration, especially in companies focused on performance, accessibility, and SEO. In some cases, development teams may restructure to align with SSR workflows by for example combining frontend and backend responsibilities in vertical product teams. [1][4][6]

4.2 Frameworks

The evolution of JavaScript frameworks has profoundly impacted how SSR is approached in modern web development, offering structured abstractions, tooling, and best practices that simplify and enhance the process of building server-rendered applications. Frameworks like Next.js, Nuxt, SvelteKit, Angular, and Astro have not only made SSR more accessible, but have also expanded its capabilities, making it a viable default for many use cases.

4.2.1 Key features and effects on development

At the core, these frameworks provide built-in SSR capabilities that abstract away much of the boilerplate required in a custom SSR setup. Their key features include:

- Automatic routing and code splitting, which optimize load times and maintainability.
- Hybrid rendering capabilities, allowing developers to choose on a per-route or per-component basis whether content should be rendered server-side, statically, or on the client.
- Integrated data fetching methods (e.g., getServerSideProps in Next.js or load in SvelteKit), which are context-aware and optimized for SSR scenarios.
- Built-in development servers and hot module reloading, simplifying the local development experience.
- Out-of-the-box hydration handling, minimizing the risk of content mismatch errors between server-rendered HTML and client-side JavaScript.

These features collectively reduce the complexity of SSR while enabling developers to write more declarative and modular code. They also introduce conventions that encourage best practices, e.g., performance-wise such as lazy loading, tree shaking, and caching. [2][3][25][26][27][28][29][30][33][35][36][39][42]

4.2.2 Differences between frameworks

Although all of these frameworks support SSR, their approaches and philosophies differ.

Next.js aims to be a complete package for projects and products of all sizes. It emphasizes full-stack capabilities with robust API routes, middleware, and a hybrid ISR/SSR/CSR model. It tightly integrates with Vercel's own hosting and edge functions, making it an ideal option for hosting large-scale React applications with performance needs. Parties with extensive hosting possibilities on their behalf can also skip this and rely on their own hosting, should they choose to do so. [2][3][5][6]

Nuxt is much like Next.js, as it takes a similar approach for the Vue ecosystem, offering file-based routing, server middleware, and full support for SSR and SSG with recent versions. The main difference between Nuxt and Next.js is the underlying framework, which is Vue in Nuxt as opposed to React in Next.js. Both of them serve as a fully-fledged solution for developing modern web software applications, and choosing between them should most likely rely on the preference of the underlying framework/library. In other words, if the development team is more familiar with Vue, they should choose Nuxt and vice versa. [2][3][28][29]

SvelteKit brings SSR to the Svelte ecosystem with a strong emphasis on compiler-based optimizations. Its *app/paths* and *app/environment* modules allow clean separation between server and client code. Its minimalist philosophy results in smaller bundles and faster execution, but might require a greater understanding of the Svelte's reactivity model compared to React or Vue. Svelte and SvelteKit function as an alternative for React and Vue, Next.js and Nuxt, offering a more lightweight, compiler-driven approach to building reactive user interfaces and server-rendered applications. [30][31][33]

Angular's SSR functionality is available directly in the framework and therefore may function as a more complete package compared to the previous three, as they build on top of existing libraries and frameworks. However, due to Angular's opinionated and monolithic nature, its SSR capabilities are more rigid and enterprise-focused. The learning curve for experienced Angular developers should be very straightforward, as the SSR technology is included in Angular's base packages, making it easy to enable in Angular projects. [2][3][35][36][38]

Astro stands out from the other frameworks as it works with the other frameworks. It can use components from multiple frameworks (React, Vue, Svelte, etc.) and render them on the server and only hydrates parts that are configured to do so (this is the "islands architecture" of Astro). This leads to reduced client-side JavaScript and faster load times, making it optimal for content-heavy, performance-critical sites. It also allows for maximum control of the hydration, as you can select the hydration model for each component. The components can be hydrated, e.g., on site load, after the site has rendered itself and entered an idle state, after a certain timeout, or once the component is visible in the browser viewing it. [1][39][40][42]

Vike sets itself also apart by offering a highly flexible approach to SSR on top of the Vite build system. Vike does not enforce a specific folder structure, routing strategy, or rendering behavior, but instead allows developers to define per-page configuration and rendering logic. This design gives teams the freedom to implement the architecture and SSR features they need. Vike supports a range of frontend libraries such as React, Vue, and others, and includes utilities for distinguishing between client and server code, customizing document structure, and opting into advanced features as needed. Although this freedom introduces a steeper learning curve and greater setup responsibility compared to more integrated solutions, it also makes Vike a

powerful tool for developers who want to retain control over more aspects of their SSR pipeline without starting from scratch. [43][44]

The approach of each framework reflects its core design goals. Next.js and Nuxt aim for full-stack, app-like flexibility. SvelteKit focuses on minimalism and reactivity. Angular targets large enterprise applications. Astro excels in performance-first, content-heavy use cases where control is crucial.

4.2.3 Framework vs. custom SSR solutions

Using a framework reduces the barrier of entry for SSR by providing standardized tooling, documentation, and ecosystem support. A custom SSR setup, which typically involves manual use of Node.js with libraries like Express, Vite, or Webpack, offers maximum control, but at the cost of increased complexity, maintenance burden, and development time. Custom setups may be necessary for highly specialized needs (e.g. custom build pipelines or legacy integrations), but for the vast majority of modern web applications, frameworks offer a more productive and error-resistant path. They handle challenges such as hydration, route-based data fetching, and cache invalidation in a tested and scalable manner, which are all common challenges in web software. [3][4][6]

4.2.4 Limitations of framework-based SSR

Despite their advantages, frameworks are not without limitations. One of them is abstraction overhead. Although helpful, abstractions can make debugging harder, especially when SSR bugs involve hydration mismatches, stale caches, or race conditions between server and client logic. Frameworks also require you to lock-in on them at least to some extent and are opionated. Astro is the exception here, as it supports the use of components created with other frameworks. Frameworks often enforce specific patterns, making it difficult to deviate from them or integrate with unconventional architectures. Migrating between frameworks often requires substantial rewrites, especially if changing technology from e.g., React to Vue or Angular. There are also performance ceilings. Default configurations may not be optimized for extreme performance scenarios or enterprise-grade deployments, requiring developers to dig into internal configurations or override defaults. On the other hand, some frameworks aim to suit, e.g., enterprise-grade deployments from the get-go. There is also limited transparency associated with frameworks. They automate a lot of processes, such as rendering pipelines, build-time optimizations, and deployment hooks, which may obscure some details from less experienced developers. Frameworks may also introduce deployment complexity, although it is not usual. Although frameworks simplify local development, the deployment of SSR applications still requires careful handling of server environments, caching strategies, and other features such as edge rendering support. Not all hosting platforms support SSR equally well, and tuning SSR performance at scale still requires expertise. [1][2][3][5][7]

In conclusion, modern JavaScript frameworks have revolutionized SSR web development by offering robust, flexible, and performance-oriented toolsets that reduce complexity and development time. Each framework brings unique strengths aligned with different priorities. These include performance (Astro), interactivity (Next.js, Nuxt), minimalism (SvelteKit), or enterprise features (Angular). Although frameworks simplify SSR significantly compared to custom solutions, they introduce new forms of abstractions and dependencies that developers must understand and navigate. Mastery of these tools, along with an appreciation of their trade-offs, is essential to build performant and maintainable server-rendered web applications.

4.3 Migration

Migrating a web application from a CSR architecture to SSR is increasingly common as developers seek to improve performance, SEO, and TTI. Although technically possible in most modern frameworks, the ease and effectiveness of such a migration depend on the original architecture, the size and complexity of the codebase, and the goals of the migration. The process is neither trivial nor universally applicable, but with the right strategy, it can be feasible and highly rewarding.

In most cases, migrating from CSR to SSR is possible, especially when the application is built with frameworks or libraries like React, Vue, Svelte, and Angular, which support both rendering paradigms. Frameworks like Next.js, Nuxt, SvelteKit, Angular, Astro and Vike are specifically designed to allow developers to transition from CSR to SSR or to adopt a hybrid strategy. However, the difficulty of migration varies widely and from framework to framework. [6]

CSR applications are designed to load minimal HTML and rely heavily on JavaScript to dynamically render content in the browser. By contrast, SSR requires rendering the application on the server before sending the fully-formed HTML to the client. This fundamental change can expose assumptions made in CSR development, such as unrestricted access to browser-only APIs (window, document, localStorage) or reliance on asynchronous side effects during component mounting, that can break or misbehave in an SSR context.

Migration is generally easier when the CSR application is modular, uses a framework that supports SSR, and has a clear separation between presentation, data fetching, and logic. For instance, migrating a well-structured React SPA to Next.js can be relatively straightforward if routing and data fetching are decoupled from components (although many blockers exist) [8]. Similarly, Vue apps that avoid tightly coupled UI logic and stateful global dependencies are more portable to Nuxt. The easiest way to migrate is to have the software built with Next.js, Nuxt, Angular, SvelteKit, Astro, or Vike and handle the migration via the frameworks' utilities.

Understandably, this might not be the case for many applications that would need to

adopt SSR. In this case, using a meta-framework like Astro or Vike can be a valid solution as one can use components made with other frameworks, and Astro or Vike would function as scaffolding software in that instance. Scaffolding the software with either of those creates a solid foundation for the application, from where you can start to adopt the existing components one by one. Both of them can also be used to incrementally to transition to SSR as one can determine, e.g., page by page, what is rendered on the server. [8][40]

On the other hand, migration may prove to be difficult when the application has:

- Heavy reliance on browser-specific APIs without context checks, requiring a refactor to ensure code does not break during server-side execution.
- Complex global state management that is built with only browser in mind, making it harder to synchronize initial server-rendered state with client-side hydration.
- Routing that is tightly integrated with component logic, making it potentially harder to adapt to file-based or server-aware routing systems in SSR frameworks.
- Non-modular architecture or legacy codebases
- Authentication flows and session handling designed purely for the client, which often need to be modified to work on the server.

[8]

Migration to SSR can also be technically challenging if the team, e.g. lacks experience with server-side environments, HTTP caching strategies, or workflows that accommodate server-rendered pages. [4]

Whether migrating to SSR is worth the effort depends on the goals of the application and the current performance, user experience, and visibility constraints. SSR offers clear benefits, such as:

- Improved SEO: Since content is rendered on the server, crawlers can index the site more effectively than CSR apps, which may require dynamic rendering or pre-rendering for search engines to see actual content.
- Faster initial page loads: SSR delivers fully-rendered HTML on first load, which can improve metrics like FCP, TTFB, and TTI.
- Improved accessibility: Because SSR provides usable content even without JavaScript enabled, it can improve accessibility.
- Better perceived performance, especially on low-end devices and slow networks, since users see content earlier.

[2][3][5][7][8]

However, these benefits come with trade-offs. SSR introduces complexity in deployment, increases server load, complicates caching and state management, and demands careful hydration to ensure consistency between server-rendered and client-interactive states. If the application is primarily a highly interactive dashboard or a tool used after authentication (e.g., internal tools), the benefits of SSR may be negligible compared to the cost of migration. [4][6][8]

Migrating can be seen as beneficial if:

- The application is content-heavy and benefits from SEO.
- Performance on initial load is a priority.
- The app is public-facing and must work well across various network and device conditions.

Migrating may not be worth the effort if:

- The application is already optimized for CSR and performs well.
- The added complexity and workload would outweigh the benefits for the use case.
- The application is highly dynamic and does not benefit meaningfully from SSR.

[3][6][7][8]

4.3.1 Migration checklist

Potential best practices for migrating to SSR include:

- Migrating within framework: Migration will be easiest when done within the same framework without as massive overhauls as it could otherwise require.
- Meta-frameworks: Meta-frameworks, like Astro and Vike, can generally be considered one of the best migration paths as they can work well with existing software.
- Incremental adoption: Use frameworks that support hybrid rendering and begin by enabling SSR on only a few pages, typically the most SEO-critical or traffic-heavy ones.
- Isomorphic code: Aim for universal (isomorphic) JavaScript that can run in both environments without branching logic where possible.
- Optimize caching and performance: SSR adds server cost. This can be compensated with strategies like full-page caching, Incremental Static Regeneration (ISR), or edge rendering to keep performance and scalability in check.

– Revise authentication: Adjust client-only authentication to include secure server-side session handling or token validation mechanisms.

[4][8]

In conclusion, migrating from CSR to SSR is possible and should be easier with modern frameworks and tooling than with implementing a custom solution. Although it introduces challenges, especially in terms of code rewriting need, the potential gains in SEO, performance, and user experience can be seemed as worth the effort in some situations. Migration should be approached thoughtfully, ideally incrementally, and with an understanding of the trade-offs involved. The process relies heavily on the application in question, as its technology choices largely define whether the migration can be done with the current foundation or if a complete overhaul or rescaffolding with another framework would be needed. When done correctly, it can significantly elevate the quality and accessibility of a web application.

4.3.2 Migration example from CSR to SSR

Here is an example of a migration from a React CSR application that is created with Vite to an SSR application with Vike. These technologies were chosen for their popularity and ease of migration. The goal is to demonstrate the key differences between the two rendering strategies with a minimalistic setup. The steps are based on Vike's documentation and GitHub [45][46]. The process expects the computer system to have Node.js, NPM, React, and Vite installed, and that the user installs the needed dependencies as they are prompted during the process.

1. Initial CSR Setup (Vite + React)

Start with a simple CSR app scaffolded using Vite (version ^6.3.5) and React:

```
npm create vite@latest my-csr-app --template react
cd my-csr-app
npm install
```

Create a basic React component representing our application page (e.g., src/pages/Home.jsx):

```
// src/pages/Home.jsx
import { useState } from "react";

export default function Home() {
  const [time, _setTime] = useState(new Date().toTimeString());
  return (
    <div>
      <h1>Hello from CSR</h1>
      <p>The current time is: {time}</p>
    </div>
  );
}
```

Edit the main entry point (CSR) to include our home page component:

```
// src/main.jsx
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import Home from './pages/Home'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <Home />
  </StrictMode>
)
```

At this stage, the application is client-side rendered and can be run like a normal application via NPM.

2. Migrate from Vite to Vike and from CSR to SSR

Install Vike and necessary dependencies by running the following script:

```
npm install vike react react-dom express cross-env
```

The versions for each package should be (other versions might work too, but these are the versions used in the example):

- cross-env: ^7.0.3
- express: ^5.1.0
- react: ^19.1.0
- react-dom: ^19.1.0
- vike: ^0.4.232

Update package.json to include updated scripts to be able to run the application via the Node.js server (and build using Vike's built-in scripts):

```
"scripts": {
  "dev": "npm run server:dev",
  "prod": "npm run build && npm run server:prod",
  "build": "vike build",
  "server:dev": "node ./server",
  "server:prod": "cross-env NODE_ENV=production node ./server"
},
```

Update `vite.config.js` to use Vike instead of Vite, so that it looks like this:

```
// vite.config.js
import react from '@vitejs/plugin-react'
import vike from 'vike/plugin'

export default {
  plugins: [react(), vike()],
}
```

Update the project structure (add Express server, page and rendering hooks):

```
src/
|- pages/
|   |- index/
|       |- +Page.jsx
|- renderer/
|   |- +onRenderClient.js
|   |- +onRenderHtml.js
server/
|- index.js
|- root.js
```

3. Create the SSR Page

The page is similiar to the earlier Home page:

```
// src/pages/index/+Page.jsx
export default function Page() {
  const time = new Date().toTimeString();
  return (
    <div>
      <h1>Hello from SSR</h1>
      <p>Server-rendered time: {time}</p>
    </div>
  );
}
```

4. Define the renderer

Here we add rendering hooks that are a common way to control the rendering behavior in Vike.

```jsx
// src/renderer/+onRenderClient.jsx
import { hydrateRoot } from 'react-dom/client'

export { onRenderClient }

async function onRenderClient(pageContext) {
  const { Page } = pageContext
  hydrateRoot(
    document.getElementById('root'),
    <Page />
  )
}
```

```jsx
// src/renderer/+onRenderHtml.jsx
export { onRenderHtml }

import { renderToString } from 'react-dom/server'
import { escapeInject, dangerouslySkipEscape } from 'vike/server'

async function onRenderHtml(pageContext) {
  const { Page } = pageContext
  const pageHtml = dangerouslySkipEscape(
    renderToString(
      <Page />
    ),
  )

  return escapeInject`
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <div id="root">${pageHtml}</div>
  </body>
</html>
`
}
```

5. Set up the Express server

The code defines a basic Express-based Node.js server that renders the page upon request.

```
// server/index.js
import express from 'express'
import { renderPage, createDevMiddleware } from 'vike/server'
import { root } from './root.js'

const isProduction = process.env.NODE_ENV === 'production'

startServer()

async function startServer() {
  const app = express()

  if (!isProduction) {
    const { devMiddleware } = await createDevMiddleware({ root })
    app.use(devMiddleware)
  } else {
    app.use(express.static(`${root}/dist/client`))
  }

  app.get('*splat', async (req, res) => {
    const pageContext = await renderPage({ urlOriginal: req.originalUrl })
    const { body, statusCode, headers } = pageContext.httpResponse
    headers.forEach(([name, value]) => res.setHeader(name, value))
    res.status(statusCode).send(body)
  })

  const port = process.env.PORT || 3000
  app.listen(port, () => {
    console.log(`App listening on port ${port}`)
  })
}
```

```
// server/root.js
export { root }

import { dirname } from 'path'
import { fileURLToPath } from 'url'
const __dirname = dirname(fileURLToPath(import.meta.url))
const root = `${__dirname}/..`
```

6. Build and run the application

Use the scripts defined in `package.json` to build and start the application:

```
npm run dev
```

or (for production mode)

```
npm run prod
```

7. Final notes

After implementing the changes and running the application with the Express server, the updated behavior can be verified by looking at the server response when accessing the page. The initial server response contains a fully rendered HTML file instead of stub HTML with the body consisting of a script tag pointing to `main.jsx`.

What changed from CSR to SSR:

- Instead of rendering entirely on the client, the HTML is now fully rendered on the server.
- The Page component still uses standard React, but is now rendered before being sent to the browser.
- An Express server now serves the page instead of a development server that is integrated in Vite or Vike

Why Vike works well for this example:

- It shows the internals of SSR without hiding them behind too heavy abstractions.
- It works with multiple frameworks, but does not enforce any.
- It is based on Vite, making the setup fast and familiar.

This example demonstrates that migrating from CSR to SSR is possible with minimal changes when using modular and modern tooling. Frameworks like Vike balance simplicity and control, making them ideal for learning and incremental adoption.

5 Conclusions

This thesis has explored the evolving landscape of web development through the lens of rendering strategies, with a particular emphasis on SSR and its growing significance in development of modern applications. By investigating the architectural paradigms of CSR, SSR, and hybrid models, the thesis has highlighted how the ability to choose between rendering strategies has reshaped how web software is designed, built, and deployed, allowing for more solution options, depending on the needs. One of the central findings is that SSR can significantly enhance performance and search engine optimization by reducing time-to-content and delivering HTML directly from the server, which beneficial for content-heavy, publicly indexed websites. At the same time, SSR introduces complexities in state management, dual-context execution (i.e., shared logic between server and client), and data fetching strategies that developers must carefully account for.

The thesis has also provided a comparative analysis of modern JavaScript frameworks such as Next.js, Nuxt, SvelteKit, Angular, and Astro, each of which offers unique abstractions and strengths in SSR-oriented development. Next.js and Nuxt are particularly noted for their robust utilities, conventions, and built-in hybrid rendering capabilities, while SvelteKit provides a reactive and lightweight approach that reduces JavaScript payloads. Astro introduces an alternative solution by shipping minimal client-side JavaScript, emphasizing partial hydration and component-level SSR, functioning as a meta-framework like Vike. The discussion emphasizes that although using such frameworks simplifies SSR development considerably, they also introduce framework-specific conventions and limitations that must be understood when making architectural decisions. This is contrasted with custom SSR implementations using tools like Vite or Express, which offer flexibility but require further implementation in areas such as routing, hydration, and deployment.

Another critical part of the thesis has been the examination of migration from CSR to SSR, a path many projects consider in the pursuit of performance and SEO gains. The migration is often technically possible but not always trivial. Its complexity largely depends on how the original CSR application has been implemented and the technologies behind it. The thesis presents potential best practices such as isolating server/client-specific logic and gradually introducing SSR using hybrid frameworks. A case study using Vike demonstrates how a CSR Vite app can be transformed into an SSR-capable application with relatively low effort.

Finally, the thesis reflects on the future of SSR within web development. As frameworks continue to evolve toward more intelligent rendering strategies, the line between CSR and SSR is increasingly blurred. Concepts like streaming, edge rendering, and partial hydration point toward a future where the server-client boundary is more fluid and optimized per use case. Ultimately, this thesis argues that understanding the capabilities, trade-offs, and properties of SSR is very valuable and a clear asset for modern web developers aiming to build high-quality applications.

While this thesis outlines general principles, comparisons, and potential pitfalls, it is important to acknowledge that the practical implications of these findings can vary depending on the specific technologies and frameworks in use. Each SSR-capable tool comes with its own design philosophies that influence how SSR is implemented and maintained. These unique characteristics can greatly affect the difficulty of migration, the performance benefits gained, possible safety concerns, or the architectural constraints. Therefore, while the study highlights broad areas that are affected by SSR and demand attention, its goal is not to label the adoption of SSR as overly risky. Instead, it aims to equip developers with a framework for critical analysis, encouraging careful evaluation of the SSR tools they choose and attention to the fine details.

5.1 Future work

Future work could continue from where this thesis ends, ideally incorporating real-world experience of large-scale SSR development processes, where the development team has experience in developing both CSR and SSR applications, to re-evaluate findings presented in this thesis. Another aspect that would benefit from further research is testing in SSR applications, as it does not seem to be very thoroughly researched yet. Future work could also focus on exploring emerging frontend frameworks such as SolidJS and Qwik, which represent promising shifts in how web applications are built and optimized. SolidJS emphasizes fine-grained reactivity without a virtual DOM, resulting in fast performance and minimal runtime overhead. Its design encourages more direct control over component behavior while maintaining a developer experience similar to React. Qwik introduces the concept of resumability, allowing applications to delay the loading and execution of JavaScript until necessary. This enables faster page loads and better performance, especially on lower-end devices or poor network conditions. Resumability has already been researched by, e.g. Juho Vepsäläinen et. al. in part V of his doctoral theses, *Resumability—A New Primitive for Developing Web Applications* [1]. Both frameworks are still relatively young, but their unique approaches to rendering, reactivity, and performance optimization suggest a strong potential for future adoption. Further research could investigate their maturity, developer experience, and how well they scale in complex applications compared to more established tools like React, Vue, Svelte, or Angular.

References

[1] J. Vepsäläinen, *Emergence of hybrid rendering models in web application development*, Doctoral thesis, 2025. Avaliable https://urn.fi/URN:ISBN:978-952-64-2486-6 (accessed on 24.6.2025).

[2] N. S. Rao, *Modern Server-Side Rendering: A Technical Deep Dive*, Article, 2025. Available https://doi.org/10.34218/IJRCAIT_08_01_059 (accessed on 25.6.2025).

[3] K. Vallamsetla, *The Impact of Server-Side Rendering on UI Performance and SEO*, Article, 2025. Available https://doi.org/10.34218/IJRCAIT_08_01_059 (accessed on 25.6.2025).

[4] A. Huotala, *Benefits and Challenges of Isomorphism in Single-Page Applications: A Case Study and Review of Gray Literature*, Master's thesis, 2021. Available http://hdl.handle.net/10138/330779 (accessed on 25.6.2025).

[5] C. Nordström, A. Dixelius, *Comparisons of Server-side Rendering and Client-side Rendering for Web Pages*, Dissertation, 2023. Available https://urn.kb.se/resolve?urn=urn:nbn:se:uu:diva-511655 (accessed on 1.3.2025).

[6] T. Gamma, N. Gerasimenko, *Swisscom Design System - Server-Side-Rendering*, Thesis (other), 2024. Available https://eprints.ost.ch/id/eprint/1175/ (accessed on 18.6.2025).

[7] O. Lyxell, *Server-Side Rendering in React: When Does It Become Beneficial to Your Web Program?*, Dissertation, 2023. Available https://urn.kb.se/resolve?urn=urn:nbn:se:umu:diva-209391 (accessed on 18.6.2025).

[8] J. Soitinaho, *Migrating Enterprise Single-Page Application to Server-side Renderable Application*, Master's thesis, 2024. Available https://urn.fi/URN:NBN:fi:aalto-202501302222 (accessed on 25.6.2025)

[9] MDN Web Docs, *Introduction to the DOM*. [Online]. Available https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction (accessed on 1.3.2025).

[10] MDN Web Docs, *Client-side rendering (CSR)*. [Online]. Available https://developer.mozilla.org/en-US/docs/Glossary/CSR (accessed on 3.3.2025).

[11] Next.js, *Client-side Rendering (CSR)*. [Online]. Available https://nextjs.org/docs/pages/building-your-application/rendering/client-side-rendering (accessed on 3.3.2025).

[12] Patterns, *Client-side Rendering*. [Online]. Available https://www.patterns.dev/react/client-side-rendering/ (accessed on 3.3.2025).

[13] Geeks For Geeks, *Server Side Rendering vs Client Side Rendering vs Server Side Generation*. Available https://www.geeksforgeeks.org/server-side-rendering-vs-client-side-rendering-vs-server-side-generation/ (accessed on 5.3.2025).

[14] Prismic, *What is Client-side Rendering (CSR)?*. [Online]. Available https://prismic.io/blog/client-side-rendering (accessed on 5.3.2025).

[15] MDN Web Docs, *Server-side rendering (SSR)*. [Online]. Available https://developer.mozilla.org/en-US/docs/Glossary/SSR (accessed on 7.3.2025).

[16] MDN Web Docs, *Static site generator (SSG)*. [Online]. Available https://developer.mozilla.org/en-US/docs/Glossary/SSG (accessed on 10.3.2025).

[17] Next.js, *Static Site Generation (SSG)*. [Online]. Available https://nextjs.org/docs/pages/building-your-application/rendering/static-site-generation (accessed on 11.3.2025).

[18] Digital Ocean, *An Introduction to Static Site Generators*. [Online]. Available https://www.digitalocean.com/community/conceptual-articles/introduction-to-static-site-generators (accessed on 12.3.2025).

[19] Cloudflare, *What is a static site generator?*. [Online]. Available https://www.cloudflare.com/learning/performance/static-site-generator/ (accessed on 12.3.2025).

[20] MDN Web Docs, *JavaScript frameworks and libraries*. [Online]. Available https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries (accessed on 14.3.2025).

[21] MDN Web Docs, *Introduction to client-side frameworks*. [Online]. Available https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Introduction (accessed on 17.3.2025).

[22] React, *React*. [Online]. Available https://react.dev/ (accessed on 19.3.2025).

[23] React, *React – A JavaScript library for building user interfaces*. [Online]. Available https://legacy.reactjs.org/ (accessed on 19.3.2025).

[24] JSX. [Online]. Available https://facebook.github.io/jsx/ (accessed on 19.3.2025).

[25] Next.js, *Introduction*. [Online]. Available https://nextjs.org/docs (accessed on 19.3.2025).

[26] Next.js, *Build Your Application: Rendering*. [Online]. Available https://nextjs.org/docs/pages/building-your-application/rendering (accessed on 19.3.2025).

[27] Vue.js, *Introduction* [Online]. Available https://vuejs.org/guide/introduction.html (accessed on 21.3.2025).

[28] Nuxt, *Introduction · Get Started with Nuxt*. [Online]. Available https://nuxt.com/docs/getting-started/introduction (accessed on 21.3.2025).

[29] Vue.js, *Server-Side Rendering (SSR)*. [Online]. Available https://vuejs.org/guide/scaling-up/ssr.html (accessed on 21.3.2025).

[30] Svelte, *Overview*. [Online]. Available https://svelte.dev/docs/svelte/overview (accessed on 24.3.2025).

[31] Svelte, *Accelerating Svelte's Development*. [Online]. Available https://svelte.dev/blog/accelerating-sveltes-development (accessed on 24.3.2025).

[32] Svelte, *.svelte files*. [Online]. Available https://svelte.dev/docs/svelte/svelte-files (accessed on 24.3.2025).

[33] Svelte, *Introduction*. [Online]. Available https://svelte.dev/docs/kit/introduction (accessed on 24.3.2025).

[34] Svelte, *Glossary*. [Online]. Available https://svelte.dev/docs/kit/glossary (accessed on 24.3.2025).

[35] Angular, *What is Angular?*. [Online]. Available https://angular.dev/overview (accessed on 26.5.2025).

[36] Angular, *Server-side rendering*. [Online]. Available https://angular.dev/guide/ssr (accessed on 26.5.2025).

[37] GeeksForGeeks, *Server-Side Rendering in Angular*. [Online]. Available https://www.geeksforgeeks.org/server-side-rendering-in-angular/ (accessed on 26.5.2025).

[38] Medium, *Replacing Angular Universal with SSR version 17.0*. [Online]. Available https://medium.com/@aayyash/replacing-angular-universal-with-ssr-version-f257dfae305f (accessed on 26.5.2025).

[39] Astro, *Why Astro? | Docs*. [Online]. Available https://docs.astro.build/en/concepts/why-astro/ (accessed on 27.5.2025).

[40] Astro, *Front-end frameworks | Docs*. [Online]. Available https://docs.astro.build/en/guides/framework-components/ (accessed on 27.5.2025).

[41] DevInterface, *Astro: everything you need to know about this increasingly popular framework*. [Online]. Available https://www.devinterface.com/en/blog/astro-everything-about-framework (accessed on 27.5.2025).

[42] Astro, *On-demand rendering | Docs*. [Online]. Available https://docs.astro.build/en/guides/on-demand-rendering/ (accessed on 27.5.2025).

[43] Vite, *Vite | Next Generation Frontend Tooling*. [Online]. Available https://vite.dev/ (accessed on 10.6.2025).

[44] Vike, *Vike*. [Online]. Available https://vike.dev (accessed on 10.6.2025).

[45] Vike, *Add SSR/SSG to existing Vite app*. [Online]. Available https://vike.dev/add (accessed on 17.6.2025).

[46] Github (@brillout), *Add SSR/SSG to an existing Vite app*. [Online]. Available https://github.com/brillout/vite-to-vike (accessed on 17.6.2025).

[47] StudyRaid, *Common State Management Challenges in SSR*. [Online]. Available https://app.studyraid.com/en/read/11947/381032/common-state-management-challenges-in-ssr (accessed on 25.6.2025).

[48] StudyRaid, *Sensitive Data Handling*. [Online]. Available https://app.studyraid.com/en/read/11947/381058/sensitive-data-handling (accessed on 25.6.2025).

[49] Medium, *How Do You Debug SSR React Applications? Step-by-Step Solutions Explained*. [Online]. Available https://medium.com/@conboys111/how-do-you-debug-ssr-react-applications-step-by-step-solutions-explained-f8173f8ae578 (accessed on 25.6.2025).